

# P1279R0

## std::breakpoint

Published Proposal, 2018-10-05

**This version:**

<https://wg21.link/p1279r0>

**Author:**

[Isabella Muerte](#)

**Audience:**

SG14, LEWG

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Current Render:**

[P1279](#)

**Current Source:**

[slurps-mad-rips/papers/proposals/breakpoint.bs](http://slurps-mad-rips/papers/proposals/breakpoint.bs)

**Implementation:**

[slurps-mad-rips/breakpoint](http://slurps-mad-rips/breakpoint)

---

## Abstract

A builtin standard breakpoint function would aid in software development.

# Table of Contents

## 1 Revision History

### 1.1 Revision 0

## 2 Motivation

## 3 Design

## 4 FAQ

### 4.1 Couldn't this be implemented via contracts?

### 4.2 How does this work with the upcoming stacktrace API?

## 5 Wording

### References

Informative References

## § 1. Revision History

### § 1.1. Revision 0

Initial Release 🎉

## § 2. Motivation

Setting breakpoints inside of a debugger can be difficult and confusing for newcomers to C++. Rather than having to learn C++, they have to learn a special syntax just to place a breakpoint in the exact spot they want, or rely on the interface of an IDE. At the end of the day, a simple programmer just wants to place a breakpoint so that their program stops when under the watchful eye of a debugger.

This paper proposes a new function, `std::breakpoint`, that causes a program to stop or "break" execution when it is being debugged.

## § 3. Design

The goal of the `std::breakpoint` function is to "break" when being debugged but to act as though it is a no-op when it is executing normally. This might seem difficult in practice, but nearly every platform and various debuggers supports something to this effect. However, some platforms have caveats that make implementing this "break when being debugged" behavior hard to implement correctly.

The `std::breakpoint` function is intended to go into the `<utility>` header.

## § 4. FAQ

### § 4.1. Couldn't this be implemented via contracts?

Possibly. However, a `std::breakpoint` function gives more fine grained control than "break on all contract violations".

### § 4.2. How does this work with the upcoming stacktrace API?

It is not intended to interoperate with the stacktrace API at this time. Both the stacktrace API and `std::breakpoint` are essentially orthogonal in their focus and use. Whereas the former is for getting more information in the event of an error, `std::breakpoint` is intended to help programmers inspect the current state of a program at a breakpoint.

## § 5. Wording

Wording is relative to [\[N4762\]](#)

```
namespace std {  
    void breakpoint(.) noexcept;  
}
```

### 1. Remarks

When this function is executed, it first must perform an implementation defined check to see if the program is currently running under a debugger. If it is, the program's execution is temporarily halted and execution is handed to the debugger until such a time as:

- the program is terminated by the debugger or,
- the debugger hands execution back to the program.

## § References

### § Informative References

#### [N4762]

Richard Smith. Working Draft, Standard for Programming Language C+. 7 July 2018. URL: <https://wg21.link/n4762>